Rapport

Application de Gestion de Dossiers Patients

Cabinet d'Ophtalmologie

Table des matières

1 Vue d'ensemble du projet **2**

2 Architecture globale **2**
- 2.1 Séparation extraction / interface . . . 2
- 2.2 Flux de données . . . 2

3 Traitement de données avec Pandas **2**
- 3.1 Chargement et inventaire . . . 2
- 3.2 Gestion des dates (`dayfirst`) . . . 2
- 3.3 Nettoyage des valeurs nulles . . . 3
- 3.4 Normalisation des identifiants . . . 3
- 3.5 Jointures directes et indirectes . . . 3

4 Algorithme de recherche robuste **4**
- 4.1 Pipeline de normalisation Unicode . . . 4
- 4.2 Propriétés de l'algorithme . . . 4

5 Génération PDF (fpdf2) **4**
- 5.1 Gestion des polices Unicode . . . 4
- 5.2 Nettoyage des caractères typographiques . . . 5
- 5.3 Structure du document généré . . . 5
- 5.4 Export partiel par consultation . . . 5

6 Interface Streamlit **5**
- 6.1 Cache et performances . . . 5
- 6.2 Rendu OD/OG . . . 5
- 6.3 CSS personnalisé . . . 5

7 Dépendances **6**

Vue d'ensemble du projet

L'application est un outil interne développé en Python, conçu pour consolider et restituer des dossiers médicaux patients depuis un ensemble de fichiers JSON hétérogènes. Elle s'appuie sur deux composants principaux :

— **`extraction.py`** : couche de données (chargement, nettoyage, jointures).
— **`app.py`** : interface utilisateur Streamlit, rendu et génération PDF.

Cette séparation garantit l'indépendance entre la logique métier et la présentation, facilitant la maintenance et les tests unitaires de chaque couche.

Architecture globale

Séparation extraction / interface

Module	Fichier	Responsabilités
Couche données	`extraction.py`	Chargement JSON, nettoyage, normalisation des IDs, jointures multi-fichiers
Interface UI	`app.py`	Recherche, rendu HTML/CSS Streamlit, tri, génération PDF
Cache ressource	`app.py`	`@st.cache_resource` sur `init_data()` pour éviter les rechargements

Flux de données

1. Au démarrage, `load_all_data()` scanne `data_raw/*.json` et charge chaque fichier en `DataFrame` Pandas dans un dictionnaire clé = nom de fichier.
2. La recherche déclenche `get_full_patient_record()`, qui localise le patient dans `Patients.json`, puis suit les liens d'ID vers les tables satellites (`Consultation`, `Documents`, `tKERATO`, `tREFRACTION`).
3. L'interface reçoit un dictionnaire `{section: DataFrame}` et orchestre l'affichage par onglets.

Traitement de données avec Pandas

Chargement et inventaire

```python
def load_all_data(data_folder: str = "data_raw") -> dict:
    base_path = Path(data_folder)
    dfs = {}
    for file_path in base_path.glob("*.json"):
        dfs[file_path.stem] = pd.read_json(file_path)
    return dfs
```

Le chargement est générique : tout nouveau fichier JSON déposé dans `data_raw/` sera automatiquement intégré sans modification du code.

Gestion des dates (`dayfirst`)

Les dates stockées dans les JSON suivent le format européen `DD/MM/YYYY`. La fonction `to_datetime_safe()` gère la double compatibilité avec les versions récentes et anciennes de Pandas :

```python
def to_datetime_safe(series: pd.Series) -> pd.Series:
    try:
        return pd.to_datetime(series, errors="coerce",
                              format="mixed", dayfirst=True)
    except ValueError:
        return pd.to_datetime(series, errors="coerce", dayfirst=True)
```

Le paramètre `errors="coerce"` transforme les valeurs non parsables en `NaT`, évitant toute exception sur des données corrompues.

Nettoyage des valeurs nulles

La fonction `clean_df()` normalise les chaînes pseudo-nulles avant de supprimer les colonnes entièrement vides :

```python
NULL_STRINGS = [r"^\s*$", "NaN", "nan", "null", "None", ""]
df[str_cols] = df[str_cols].replace(NULL_STRINGS, np.nan, regex=True)
df = df.dropna(axis=1, how="all")
```

Côté interface, `is_empty()` complète cette logique en couvrant `None`, `float('nan')`, `pd.NaT` et les chaînes vides.

Normalisation des identifiants

Les IDs transitent entre fichiers sous des types hétérogènes (`int`, `float`, `str`). `normalize_id()` les ramène systématiquement à un entier stringifié :

```python
def normalize_id(value) -> str | None:
    if value is None or (isinstance(value, float) and np.isnan(value)):
        return None
    try:
        return str(int(float(str(value).strip())))
    except (ValueError, OverflowError):
        s = str(value).strip()
        return s if s else None
```

Ceci résout le problème classique de comparaison `"42.0" != "42"` lors de jointures cross-fichiers.

Jointures directes et indirectes

Liens directs (Code patient → table) :

```python
direct_links = {
    "Ag_Rdv":       "Code Patient",
    "Consultation": "Code patient",
    "Documents":    "code patient",
    "tPostIT":      "CodePat",
}
for section, id_col in direct_links.items():
    mask = df[id_col].apply(normalize_id) == patient_id
    record[section] = clean_df(df[mask])
```

Liens indirects (via numéros de consultation, pour `tKERATO` et `tREFRACTION`) :

```python
consult_ids = record["Consultation"]["N   consultation"] \
              .apply(normalize_id).dropna().tolist()
mask = df["NumConsult"].apply(normalize_id).isin(consult_ids)
```

Le tri chronologique des examens techniques passe par une jointure temporaire sur la date de consultation :

```
def sort_exams_via_consult(exam_df, consult_df, ...):
    date_lookup = consult_df[[consult_id_col, consult_date_col]] \
                  .rename(...).drop_duplicates(subset=[num_consult_col])
    merged = exam_df.merge(date_lookup, on=num_consult_col, how="left")
    return merged.sort_values("_date_tmp", ascending=False)
```

Algorithme de recherche robuste

Pipeline de normalisation Unicode

La recherche patient tolère les accents, les majuscules et l'ordre arbitraire des termes :

```
import unicodedata
combined_names = (nom_col + " " + prenom_col).str.lower()
combined_names = (combined_names
    .str.normalize('NFKD')
    .str.encode('ascii', errors='ignore')
    .str.decode('utf-8'))

clean_search = unicodedata.normalize('NFKD', patient_name) \
               .encode('ascii', 'ignore').decode('utf-8')
search_terms = clean_search.lower().split()

mask = pd.Series(True, index=df.index)
for term in search_terms:
    mask = mask & combined_names.str.contains(term, regex=False)
```

Propriétés de l'algorithme

- **Insensible à la casse** : tout est converti en minuscules.
- **Insensible aux accents** : décomposition NFKD puis encodage ASCII.
- **Insensible à l'ordre** : chaque terme est cherché indépendamment (ET logique).
- **Correspondance partielle** : `contains()` sans regex autorise les sous-chaînes.

Un classement par colonne de type `str` (objets Pandas) est préalablement effectué via `select_dtypes` pour éviter de normaliser des colonnes numériques.

Génération PDF (fpdf2)

Gestion des polices Unicode

La fonction `_register_fonts()` tente de charger la famille `DejaVuSansCondensed` (livrée avec `fpdf2`) pour garantir l'affichage correct des caractères accentués :

```
styles = {
    "":   "DejaVuSansCondensed.ttf",
    "B":  "DejaVuSansCondensed-Bold.ttf",
    "I":  "DejaVuSansCondensed-Oblique.ttf",
    "BI": "DejaVuSansCondensed-BoldOblique.ttf",
}
for style, filename in styles.items():
    pdf.add_font("DejaVu", style=style, fname=str(font_path), uni=True)
```

En cas d'échec (fichier manquant, version incompatible), un `except Exception` déclenche le fallback sur `Helvetica`, garantissant la génération du PDF dans tous les cas.

Nettoyage des caractères typographiques

Certains caractères Unicode courants en français ne sont pas rendus correctement par les polices PDF de base. Ils sont remplacés avant écriture :

```
safe_val = (str(val)
    .replace("\u0153", "oe")    # oe ligature
    .replace("\u2014", "-")     # tiret cadratin
    .replace("\u2019", "’")     # apostrophe typographique
    .replace("\u00ab", ’"’)     # guillemet ouvrant
    .replace("\u2026", "..."))  # points de suspension
```

Structure du document généré

Le PDF est structuré en quatre sections ordonnées : Consultations, Kératométrie, Réfraction, Documents. Chaque section est rendue via des helpers dédiés (`_pdf_section_title`, `_pdf_entry_header`, `_pdf_kv`) assurant une mise en page cohérente. Les champs vides sont silencieusement omis (`is_empty()`).

Export partiel par consultation

L’interface permet de générer un PDF ciblé sur une seule consultation. Le sous-ensemble du `record` est reconstruit à la volée en filtrant `tKERATO`, `tREFRACTION` et `Documents` par `NumConsult`, puis passé à `generate_pdf_bytes()` sans modification de la fonction de génération.

Interface Streamlit

Cache et performances

`@st.cache_resource` sur `init_data()` garantit que le chargement des JSON n’a lieu qu’une seule fois par instance de l’application, indépendamment du nombre de requêtes utilisateur.

Rendu OD/OG

La classification des colonnes d’examen repose sur leur suffixe :

```
def classify_od_og(col_name: str) -> str:
    name = col_name.strip().upper().replace("_","").replace(" ","")
    if name.endswith("OD"): return "OD"
    if name.endswith("OG"): return "OG"
    return "other"
```

Les colonnes OD et OG sont ensuite distribuées dans deux colonnes Streamlit côte à côte, avec un code couleur différenciant (bleu pour OD, vert pour OG).

CSS personnalisé

Le thème par défaut de Streamlit est surchargé via `st.markdown(..., unsafe_allow_html=True)` avec des classes CSS dédiées (`.field-label`, `.field-value`, `.eye-header`), assurant une présentation clinique uniforme indépendante du thème système de l’utilisateur.

Dépendances

Bibliothèque	Usage	Contraintes
`pandas`	Manipulation de DataFrames	`>= 1.5` (format mixed)
`numpy`	Détection NaN, normalisation	Toute version récente
`streamlit`	Interface web	`>= 1.20`
`fpdf2`	Génération PDF	`>= 2.7` (polices DejaVu)